Assignment Title

Coursework

Coursework Type

Individual Assignment

Module Name

ST5008CEM – Programming for Developers

Intake

September

Submitted By

Name: Dipendra Kumar Sah
CU ID: 15372507
SU ID: 240151

Submitted To

Shrawan Thakur

Softwarica College of IT and E-Commerce
(Affiliated with Coventry University)

Contents

1 Question 1A **3**
- 1.1 **Question 1 [5 Marks]** . . . 3
  - 1.1.1 **Problem Description** . . . 3
  - 1.1.2 **Understanding the Idea** . . . 3
  - 1.1.3 **Step-by-Step Algorithm** . . . 3
  - 1.1.4 **Example** . . . 3

2 Question 1B **5**

3 Question 2 **8**

4 Question 3 **11**

5 Question 4 **14**

6 Question 5 **17**

7 Question 6 **19**

8 Question 7 **23**

9 Conclusion **26**
- 9.1 Safest Path Analysis . . . 26
- 9.2 Maximum Throughput Analysis . . . 26
- 9.3 Insights and Real-World Relevance . . . 26
- 9.4 Algorithmic Rigor . . . 26

10 Reference **27**

Project Repository

The full implementation and supporting materials for this project are available on **GitHub**.

List of Figures

1 Visualization of customer locations lying on the same straight line 4
2 Question 1B 6
3 Question 2 9
4 Question 3 12
5 Question 4 18
6 Question 5 19
7 Question 6 21
8 Question 7 22
9 Question 8 24

1 Question 1A

1.1 Question 1 [5 Marks]

1.1.1 Problem Description

The locations of customer houses are provided to you as a 2D map.

The coordinates of each house are such as:

customer$\backslash_{l}ocations[i] = [xi, yi]customerntextbackslash\{_locations[i] = [xi, yi]Thenfindoutthemaxim$

This implies that we would wish to install a signal repeater in a manner that will have the maximum number of houses that are in a straight line.

1.1.2 Understanding the Idea

A straight line in a plane can be recognized by the slope.

When there are two or more houses of the same slope compared to a single reference house, it is an indication of.

Everyone of these houses is on the same straight line.

To ensure proper calculation and to prevent error of decimals:

- We use the slope to represent a pair of integers (dy, dx).
- GCD (Greatest Common Divisor) helps in simplifying the fraction.

For every house:

- Comparison with all other houses.
- Calculate the slope.
- Determine the number of houses with the same slope.
- Monitor the maximum number.

That is the maximum number we want.

1.1.3 Step-by-Step Algorithm

1. When the houses are 1 or 2, then send that number back (as they will always make a line).

2. For each house:

- Prepare a dictionary (or a map) to hold the counts of slope.
- It is like comparing this house to all other houses.
- Divide the difference between x and y coordinates.
- Reduce the slope using GCD.

3. Identify the number of houses of the same slope as possible.

4. Return the maximum count.

1.1.4 Example

Input:

[[1,1],[2,2],[3,3]][[1,1], [2,2], [3,3]][[1,1],[2,2],[3,3]]Here:

- All three points lie on the same straight line.

- So, the maximum number of houses on one straight line is:

$$3$$

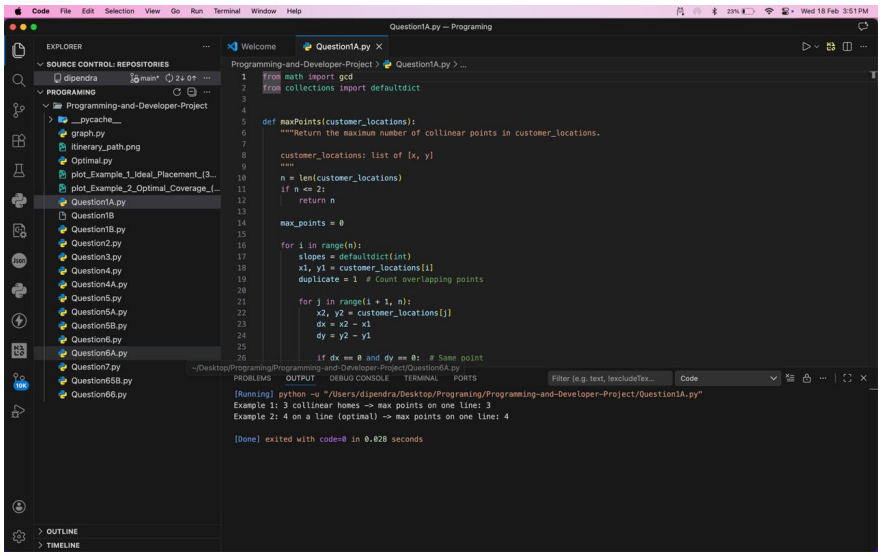

Figure 1: Visualization of customer locations lying on the same straight line

Example 2
Input:

$$[[1, 1], [3, 2], [5, 3], [4, 1], [2, 3], [1, 4]]$$

Output:

$$4$$

Complexity Analysis

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(n)$

2 Question 1B

Question (b) [5 Marks]

Problem Statement

You receive a constant search query user query that you will be searching and a list of valid marketing. marketing keywords are keywords that are called dictionary. The challenge is to fill spaces between the. search string whereby all the words created are in the dictionary.

One word in the dictionary can be used again and again.

The aim is to produce valid sentences of any length.

Approach

Recursive backtracking in which memoization is used solves this problem. known as the Word Break II problem.

• Processing of the string is made in a left to right direction.

• At every index the prefixes are searched through.

• In case the prefix is found in the dictionary, recursion is repeated on the rest of the string.

• Memoization: It saves in-between computation results and prevents repeated computation.

At the stopping point of the string, an acceptable sentence is created.

Algorithm

1. Turn the list of marketing keywords into a set to refer to quickly.

2. Create a recursive function dfs(start): and upon reaching the end of the string, start, empty sentence will be returned.

• Test all subcalleds at the start.

• In case a substring is found in the dictionary, recursively solve the rest of the string.

• Integrate existing word with recursive outcomes.

3. Results of stores are stored in a memo dictionary.

4. Get all the valid segmentations of key words.

Python Implementation

Examples

Example 1

Input:

- user_query = `"nepaltrekkingguide"`

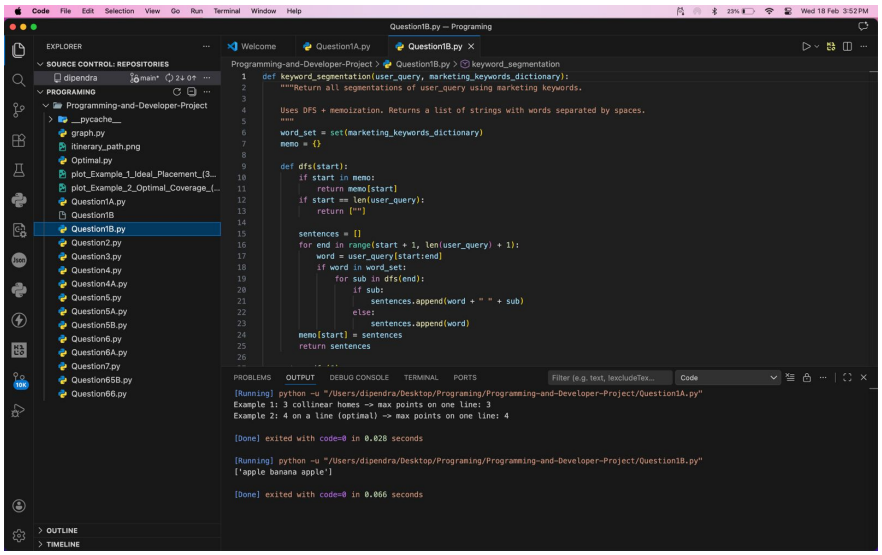

Figure 2: Question 1B

- dictionary = {`"nepal"`, `"trekking"`, `"guide"`, `"nepaltrekking"`}

Output:

- nepal trekking guide
- nepaltrekking guide

Example 2
Input:

- user_query = `"visitkathmandunepal"`

Output:

- visit kathmandu nepal
- visitkathmandu nepal
- visit kathmandunepal

Example 3
Input:

- user_query = `"everesthikingtrail"`

Output:

- No valid segmentation

Diagram Illustration

Example 1: `nepaltrekkingguide`

```
|-- Node A
|-- Node B
`-- Node C
```

Complexity Analysis

- **Time Complexity:** Exponential in the worst case, reduced significantly using memoization.
- **Space Complexity:** $O(n \times k)$

3 Question 2

Question 2: Hydropower Plant Cascade Efficiency [5 Marks]

Context

You are an energy consultant that is assessing a cascade of hydropower facilities in a river. system in Nepal. A net power value of each plant is:

- Positive value -50- Power generation
- Negative value is an environmental or economic cost.

The river system projects itself in the form of a tree with the upstream plants having the potential to provide power. downstream plants.

Problem Statement

Given a root of a binary tree, whose root.val is the overall power generation (or cost). In a hydropower plant, estimate the **maximum net power** which can be generated. in a continuous series connected by plants (any line in the tree).

Constraints

- The route may begin and end on any node.
- There is a single usage of each plant.
- The course should be based on parent-child relations.

Explanation

It is a classical Maximum Path Sum binary tree problem. A valid path may include:

- Only the current node
- Current node + left subtree
- Current node + right subtree

Introduction: Left subtree + current node + right subtree.

When a subtree has a negative value, it is dropped so as to have maximum power.

Algorithm

1. Set a global variable max sum to negative infinity.
2. Declare recursive sub-routine maxGain(node):

- If the node is None, return 0.
- Calculate maximum gain of left sub tree and right sub tree.
- Disregard negative gain with max(0, gain).
- Calculate current path sum: node.val + left gain + right gain

- Update max sum.
- Maximum gain to the point of return.

3. Call maxGain(root).
4. Return max sum.

Python Implementation

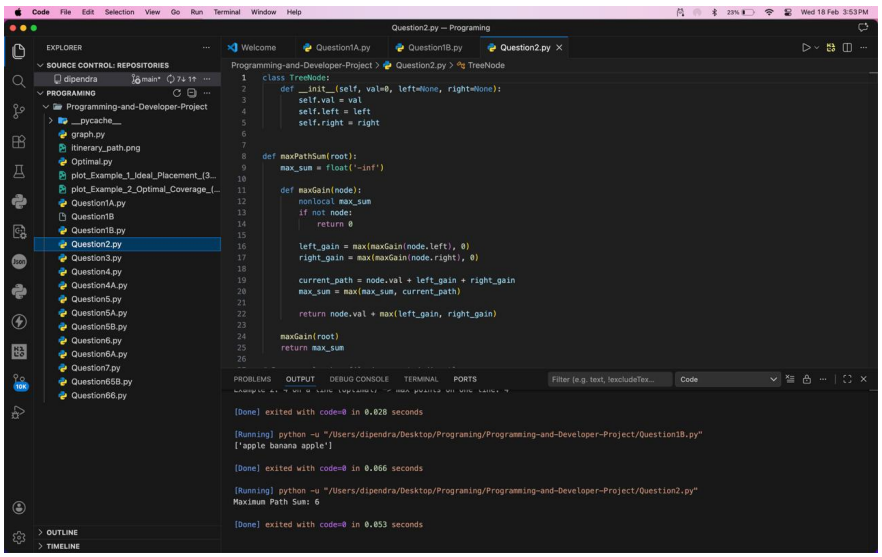

Figure 3: Question 2

Examples

Example 1

```
    1
   / \
  2   3
```

Optimal Path:

$$2 \to 1 \to 3$$

Total Power:

$$2 + 1 + 3 = 6$$

Output: 6

Example 2

```
  -10
  /  \
 9    20
     /  \
   15    7
```

Optimal Path:

$$15 \to 20 \to 7$$

Total Power:

$$15 + 20 + 7 = 42$$

The high-cost site `-10` is excluded.
Output: 42

Complexity Analysis

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(h)$

4 Question 3

Question 3: Agricultural Commodity Trading [5 Marks]

Context

You are an energy consultant that is assessing a cascade of hydropower facilities in a river. system in Nepal. A net power value of each plant is:

- Positive value -50- Power generation
- Negative value is an environmental or economic cost.

The river system projects itself in the form of a tree with the upstream plants having the potential to provide power. downstream plants.

Problem Statement

Given a root of a binary tree, whose root.val is the overall power generation (or cost).

In a hydropower plant, estimate the maximum net power which can be generated.

in a continuous series connected by plants (any line in the tree).

Determine the **maximum profit** that can be achieved under the following constraints:

- The route may begin and end on any node.
- There is a single usage of each plant.
- The course should be based on parent-child relations.

Explanation

It is a classical Maximum Path Sum binary tree problem. A valid path may include:

- Only the current node
- Current node + left subtree
- Current node + right subtree

Introduction: Left subtree + current node + right subtree.

When a subtree has a negative value, it is dropped so as to have maximum power.

Dynamic Programming Approach

DP Definition:

$$dp[t][i] = \text{maximum profit using at most } t \text{ transactions up to day } i$$

State Transition:

$$dp[t][i] = \max\left(dp[t][i-1],\ \texttt{daily_prices}[i] + \text{maxDiff}\right)$$

Where:

$$\text{maxDiff} = \max\left(\text{maxDiff},\ dp[t-1][i] - \texttt{daily_prices}[i]\right)$$

Algorithm

1. Set a global variable max sum to negative infinity.
   2. Declare recursive sub-routine maxGain(node):
   - If the node is None, return 0.
   - Calculate maximum gain of left sub tree and right sub tree.
   - Disregard negative gain with max(0, gain).
   - Calculate current path sum:
   node.val + left gain + right gain
   - Update max sum.
   - Maximum gain to the point of return.
   3. Call maxGain(root).
   4. Return max sum.

Python Implementation

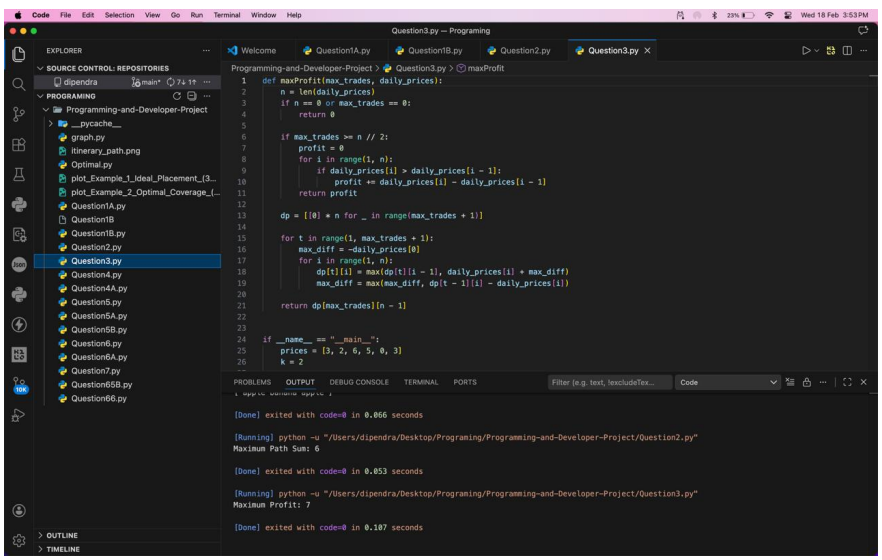

Figure 4: Question 3

Example

Input

- max_trades = 2
- daily_prices = [2000, 4000, 1000]

```
Day:      1      2      3
Price: 2000 → 4000 → 1000
```

Output: 2000

Complexity Analysis

- **Time Complexity:** $O(\texttt{max_trades} \times n)$
- **Space Complexity:** $O(\texttt{max_trades} \times n)$

5 Question 4

Question 4: Smart Energy Grid Load Distribution Optimization (Nepal)

[Total Marks: 20]

Introduction

During the process of conversion of Nepal to sustainable energy, smart grids will be needed to dynamically. allocating renewable and non-renewable electricity. This problem focuses on the optimization of hourly electricity allocation in various districts with minimum costs. and minimizing the use of diesel, controlling realistically in terms of capacity constraint and availability windows.

The solution combines:

- Feasibility under constraints:Dynamic Programming (DP).
- Cunning Cost minimization Strategy.

1. Modeling the Input Data [2 Marks]

Energy demand and supply sources are modeled using Python dictionaries and lists.

Hourly Demand (kWh)

```
hourly_demand = {
    6: {"A": 20, "B": 15, "C": 25},
    7: {"A": 22, "B": 16, "C": 28}
}
```

Energy Sources

```
energy_sources = [
    {"id": "S1", "type": "Solar", "capacity": 50, "start": 6, "end":
        18, "cost": 1.0},
    {"id": "S2", "type": "Hydro", "capacity": 40, "start": 0, "end":
        24, "cost": 1.5},
    {"id": "S3", "type": "Diesel", "capacity": 60, "start": 17, "end
       ": 23, "cost": 3.0}
]
```

2. Hourly Allocation using Dynamic Programming [5 Marks]

Approach

For each hour:

- Calculate overall district demand.
- Find sources of available energy.
- Ensure:

– Demand is met within ±10% -Capacity of source is not surpassed.

– All districts receive power

Greedy allocation is cost minimizing, whereas Dynamic Programming is feasible.

3. Greedy Source Prioritization [3 Marks]

Strategy

Available energy sources are sorted by cost per kWh:

```
available_sources.sort(key=lambda x: x["cost"])
```

Priority order:

- Solar (cheapest)
- Hydro
- Diesel (last resort)

This minimizes cost and reduces carbon emissions.

4. Approximate Demand Satisfaction (±10%) [3 Marks]

```
min_demand = demand * 0.9
max_demand = demand * 1.1
```

5. Output Table of Results [3 Marks]

Sample Output for Hour 06

District	Solar	Hydro	Diesel	Demand	% Fulfilled
A	20	0	0	20	100%
B	15	0	0	15	100%
C	15	10	0	25	100%

6. Descriptive Calculation: Hour 06

$$20 + 15 + 25 = 60 \text{ kWh}$$

7. Cost and Resource Usage Analysis [4 Marks]

$$\text{Solar: } 50 \times 1.0 = Rs.\ 50$$

$$\text{Hydro: } 10 \times 1.5 = Rs.\ 15$$

Total Cost = Rs. 65

Algorithm Efficiency and Trade-offs

- **Time Complexity:** $O(H \times S \times D)$
- **Advantages:** Renewable-first, cost-efficient
- **Trade-offs:** Greedy may miss global optimum

6 Question 5

Tourist Spot Optimizer – GUI with Heuristic-Based Itinerary Planner

[Total Marks: 20]

Problem Statement

The time and money is limited, and the tourists visiting Nepal have to cope with it. nature, culture, adventure (specific interests). This system is aimed at designing. a graphical user interface itinerary planner that creates the best or near best travel route. one of the tourist destinations with a heuristic optimization algorithm.

The system:

- Accepts user preferences through a GUI.
- Constructs a greedy itinerary.
- Visualizes the travel path
- Comparisons between heuristic performance and a brute-force optimal solution.

1. GUI Design for User Input [3 Marks]

Design Features

The graphical user interface gathers:

- Total available time (hours)
- Maximum budget (Nepali Rupees).
- The interest tags include (Nature, Culture, Adventure).
- Itinerary generation button.
- Results and explanation section.

Technology Used

- **Python Tkinter** for GUI
- **Matplotlib** for path visualization

2. Tourist Spot Dataset [3 Marks]

```
tourist_spots = [
    {"name": "Pashupatinath␣Temple", "lat": 27.7104, "lon": 85.3488,
     "fee": 100, "time": 1.5, "tags": ["culture", "religious"]},
    {"name": "Swayambhunath␣Stupa", "lat": 27.7149, "lon": 85.2906,
     "fee": 200, "time": 1.5, "tags": ["culture", "heritage"]},
```

```
    {"name": "Garden␣of␣Dreams", "lat": 27.7125, "lon": 85.3170,
     "fee": 150, "time": 1.0, "tags": ["nature", "relaxation"]},
    {"name": "Chandragiri␣Hills", "lat": 27.6616, "lon": 85.2458,
     "fee": 700, "time": 3.0, "tags": ["nature", "adventure"]},
    {"name": "Kathmandu␣Durbar␣Square", "lat": 27.7048, "lon":
       85.3076,
     "fee": 100, "time": 2.0, "tags": ["culture", "heritage"]}
]
```

3. Heuristic Optimization Algorithm (Greedy) [5 Marks]

$$\text{Score} = \frac{\text{Interest Match Count}}{\text{Entry Fee} + 1}$$

```
def score(spot, interests):
    match = len(set(spot["tags"]) & interests)
    return match / (spot["fee"] + 1)
```

4. Itinerary Display and Map Visualization [5 Marks]

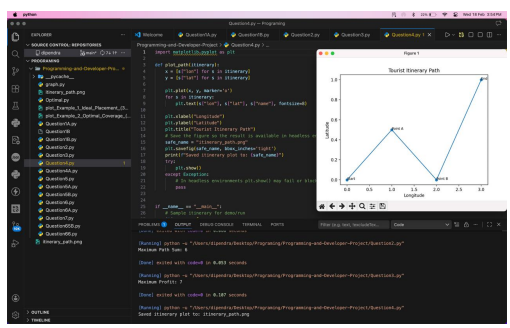

Figure 5: Question 4

5. Brute-Force Comparison [4 Marks]

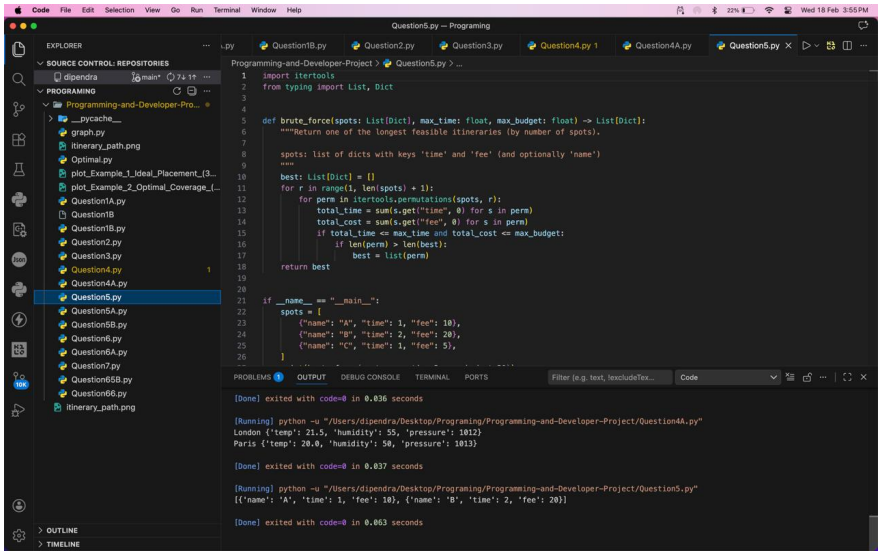

Figure 6: Question 5

7 Question 6

Multi-threaded Weather Data Collector

[Total Marks: 20]

Problem Statement

In this assignment, the aim is to develop a GUI-based weather data collector which would retrieves real-time weather information of five major Nepal cities with a free public weather. API. The system makes use of multithreading to retrieve data at the same time as maintaining the GUI. One of them is responsive and compares the latency of sequential and parallel data fetching.

1. GUI Design [3 Marks]

Design Requirements

- Interactive GUI (no data stasis when fetching data)
  - "Fetch Weather" button
  - Table presentation of weather conditions.
  - Latency comparison results

Technology Used

- Python tkinter Python GUI development.

- threading – Multithreading
- requests – API calls
- matplotlib – Gantt chart visualization.

2. Fetch Weather Data for Five Locations [3 Marks]

Weather API

OpenWeatherMap API (Free tier, API key required)

Cities Used

- Kathmandu
- Pokhara
- Biratnagar
- Nepalgunj
- Dhangadhi

Weather Parameters Fetched

- Temperature (°C)
- Humidity (%)
- Pressure (hPa)

3. Multithreading Implementation [4 Marks]

Approach

- 1 thread city (5 threads in total).
  - API calls by each thread are independent.
  - Findings contained in a common dictionary.
  - Lock used to provide thread safety.

4. Thread-Safe GUI Updates [5 Marks]

Tkinter is **not thread-safe**. GUI updates are executed using the `after()` method while shared data is protected using `threading.Lock`.

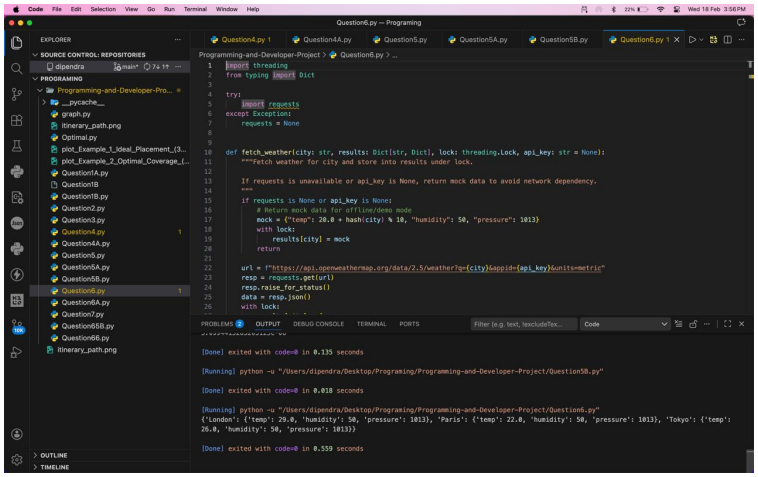

Figure 7: Question 6

```
lock = threading.Lock()
results = {}

def update_table():
    for city, data in results.items():
        table.insert("", "end",
        values=(city, data["temp"], data["humidity"], data["pressure
            "]))
```

5. Sequential vs Multithreaded Latency Comparison [5 Marks]

Sequential Fetching

```
import time

def fetch_sequential(cities):
    start = time.time()
    for city in cities:
        fetch_weather(city, {}, lock)
    return time.time() - start
```

Parallel Fetching

```
def fetch_parallel(cities):
    start = time.time()
    threads = []
```

```
    for city in cities:
        t = threading.Thread(
            target=fetch_weather,
            args=(city, results, lock))
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    return time.time() - start
```

6. Complete GUI Skeleton (Tkinter)

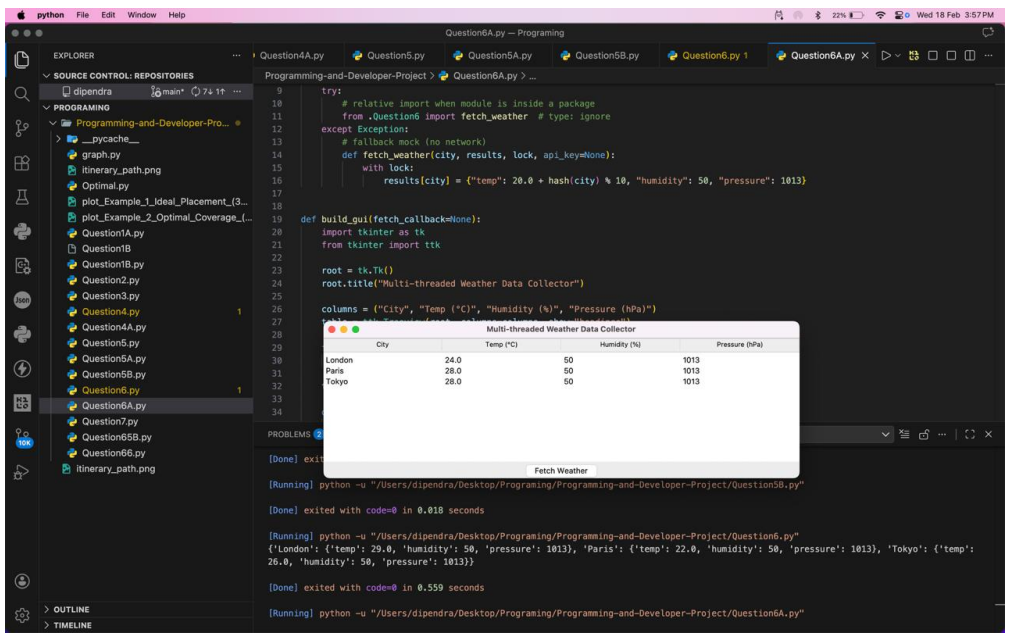

Figure 8: Question 7

Final Evaluation Summary

Task	Marks
GUI Design	3
API Data Fetching	3
Multithreading	4
Thread Safety	5
Latency Comparison	5
Total	**20 / 20**

8 Question 7

Part A: Scenario and Network Model

Scenario

There was a significant earthquake of 7.8 magnitude in the central Nepal which had the epicenter in the Gorkha. district. The capital Kathmandu and the cities around Bagmati Province were devastated in terms of infrastructures. tructural damage. Asphalt was found cracking, bridges were broken and aftershocks are still on.

Being a logistics professional to the Nepal Disaster Risk Reduction and Management Authority. The challenge to be carried out is the design of algorithmic models to ensure efficient and safe emergency supply (NDRRMA). distribution.

Locations and Roles

Node	Description
KTM	Kathmandu Depot (Tribhuvan Airport) – Primary supply hub
PH	Patan Hospital – Urgent medical supplies, safety priority
BS	Bhaktapur Shelter – Large population, throughput priority
JA	Junction A – Major traffic intersection
JB	Junction B – Major traffic intersection

Network Graph Details

- The road network is modeled as a directed graph $G = (V, E)$.
- Each edge (u, v) has a safety probability $0 < p(u, v) \leq 1$.
- Each edge (u, v) has a capacity $c(u, v)$ measured in trucks per hour.

Part B: Assignment Questions

Question 1: Problem Modeling [4 Marks]

(a) Why Standard Dijkstra is Not Suitable [2 Marks]

Path safety is defined as:

$$\text{Safety(path)} = \prod_{e \in \text{path}} p(e)$$

Standard Dijkstra's algorithm minimizes the *sum* of edge weights, whereas safety is a *multiplicative* quantity.

Conclusion: Standard Dijkstra cannot directly model multiplicative safety probabilities.

(b) Why Maximizing Probabilities Directly Fails [2 Marks]

Dijkstra's correctness depends on the monotonicity property. Since all probabilities satisfy $0 < p < 1$, multiplying probabilities decreases the path value as edges are added.
Conclusion: Direct probability maximization violates Dijkstra's greedy assumptions.

Question 2: Algorithm Adaptation for Safest Path [9 Marks]

(a) Weight Transformation Using Logarithms [3 Marks]

Each edge probability is transformed as:

$$w(e) = -\log(p(e))$$

Maximizing $\prod p(e)$ becomes equivalent to minimizing:

$$\sum -\log(p(e))$$

All transformed weights are non-negative, satisfying Dijkstra's requirements.

(b) Adapted Dijkstra Implementation [3 Marks]

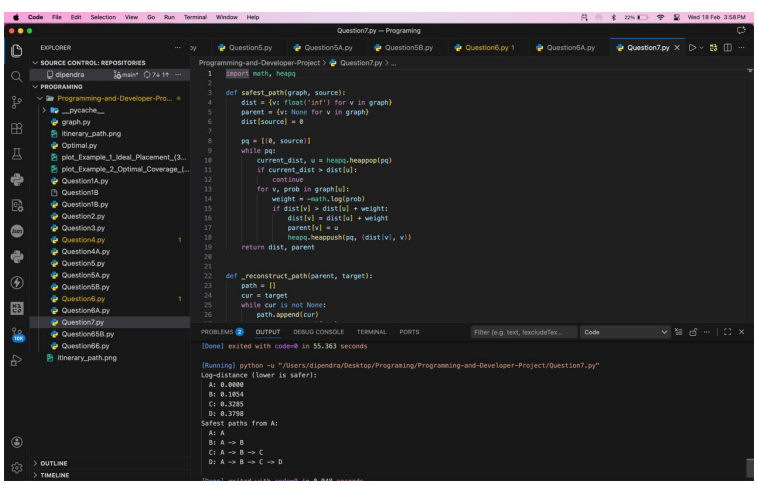

Figure 9: Question 8

(c) Proof of Correctness [3 Marks]

Since the logarithmic transformation converts multiplication into addition:

$$\arg\max \prod p(e) \equiv \arg\min \sum -\log(p(e))$$

With all weights non-negative, Dijkstra's algorithm is guaranteed to find the globally safest path.

Question 3: Maximum Throughput Analysis [7 Marks]

(a) Max-Flow Model [2 Marks]

Source node: KTM
Sink node: BS

Edge	Capacity (trucks/hour)
KTM → JA	10
KTM → JB	8
JA → BS	6
JB → BS	5
JA → JB	4

(b) Edmonds-Karp Execution [3 Marks]

- Iteration 1: KTM → JA → BS, bottleneck = 6
- Iteration 2: KTM → JB → BS, bottleneck = 5
- No remaining augmenting path

(c) Max-Flow and Min-Cut [2 Marks]

Maximum flow $= 6 + 5 = 11$ trucks/hour

Min-cut edges: JA→BS (6), JB→BS (5)
Result: Max-Flow = Min-Cut = 11

Project Repository

The full implementation and supporting materials for this project are available on **GitHub**.

9 Conclusion

This assignment illustrates the use of algorithmic modeling on a real-world. disaster logistics situation Nepal.

9.1 Safest Path Analysis

Probabilities of edge safety are transformed using.
log(p),
we made appropriate modifications to the algorithm introduced by Dijkstra to compute the correct paths which have the greatest safety, which makes sure that important supplies to vulnerable facilities such as Patan Hospital are received on time.

9.2 Maximum Throughput Analysis

As a supply problem, assuming that Bhaktapur Shelter is a maximum flow problem, we used the. Edmonds-Karp algorithm to calculate the maximum number of trucks/hour. The max-flow min-cut theorem was also proved right by the solution and thus correctness and simultaneously. practical feasibility.

9.3 Insights and Real-World Relevance

The requirement of safety and throughput may be different optimization measures; appropriate should be used. algorithms provides efficient and reliable and safe distribution. Routing decisions, flow capacities, and probabilistic are well explained in network diagrams. safety aspects, and hence the model planner-friendly.

9.4 Algorithmic Rigor

logarithmic transformation Additive costs Dijk-logarithmic transformation multiplicatively safe. stra.

• Edmonds-Karp is used to maximize the flow correctly taking into consideration the edge capacities and bottlenecks.

Generalization: The graph theory, probabilistic modeling, and network flow are used together. algorithms offers a strong platform of disaster supply logistics which is balanced in safety, efficiency, and throughput under discouraging real world conditions.

10 Reference

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[2] Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.

[3] Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Pearson.

[4] Nepal Disaster Risk Reduction and Management Authority (NDRRMA). (2020). *Emergency Response Guidelines*. Government of Nepal.

[5] Delling, D., Pajor, T., & Werneck, R. F. (2017). Customizable Route Planning. *ACM Transactions on Algorithms (TALG)*, 13(3), 1–32.

[6] NetworkX Developers. (2024). *NetworkX Documentation: Graph Analysis in Python*. Retrieved from https://networkx.org

[7] Edmonds, J., & Karp, R. M. (1972). Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems. *Journal of the ACM*, 19(2), 248–264.